BANK ONLINEX

POC — Prova de Conceito

Arquitetura de Análise de Projetos e Sistemas

Aplicação de Investimento Financeiro de Alta Transação de Dados

Desenho de arquitetura e documentação técnica UML com apoio da plataforma NotebookLM

Documento: POC-ARQ-2026-001 · Versão 1.0 · Novembro de 2026

Elaborado por: Arquitetura de Soluções — Bank OnlineX

Professor: Benir Falcão Pistille

Sumário

No Word, clique com o botão direito sobre o sumário e escolha «Atualizar Campo» para sincronizar os números de página.

Sumário 2
1. Introdução 3
1.1 Objetivo da POC 3
1.2 Escopo 3
1.3 Premissas e restrições 3
2. Requisitos da aplicação de investimentos 4
2.1 Requisitos funcionais 4
2.2 Requisitos não-funcionais (alta transação de dados) 4
3. Arquitetura proposta (TO-BE) — topologia de servidores 5
3.1 Visão geral da topologia 5
3.2 Componentes da topologia 5
3.3 Decisões de arquitetura (ADRs) 6
4. Arquitetura atual das plataformas de IA (AS-IS) 7
4.1 LLMs — Estrutura de Linguagem Natural 7
4.2 Agentes de IA — Sistemas Autônomos de Execução 7
4.3 RAG — Retrieval-Augmented Generation 8
4.4 n8n — Automação low-code de workflows 8
4.5 MCP — Model Context Protocol 8
4.6 Integração entre as plataformas 9
5. Documentação UML 10
5.1 Diagrama de Casos de Uso 10
5.2 Diagrama de Componentes 10
5.3 Diagrama de Sequência — execução de ordem 12
5.4 Diagrama de Classes (modelo de domínio) 13
5.5 Diagrama de Implantação 13
6. Metodologia: uso do NotebookLM na construção da arquitetura 14
6.1 Passo a passo da simulação no NotebookLM 14
6.2 Prompts recomendados para geração de UML 15
6.3 Boas práticas e limitações 15
7. Etapas da POC (faseamento e cronograma) 16
7.1 Fases, atividades e entregáveis 16
7.2 Critérios de sucesso (KPIs) 16
8. Riscos e mitigação 17
9. Referências e bibliografia 18

1. Introdução

1.1 Objetivo da POC

Esta Prova de Conceito (POC) tem por objetivo validar uma arquitetura de referência para a nova aplicação de Investimento Financeiro do Bank OnlineX, um banco digital com perfil de alta transação de dados (milhares de operações por segundo em horários de pico de mercado). O documento consolida o desenho da arquitetura de servidores de aplicação, web servers e demais componentes de topologia, a documentação técnica em UML (Unified Modeling Language) e o registro da arquitetura das plataformas de inteligência artificial atualmente utilizadas pela empresa.

Um diferencial metodológico desta POC é o uso da plataforma NotebookLM (Google) como ferramenta de apoio à elaboração do desenho arquitetural e da documentação UML: os artefatos de requisitos, decisões de arquitetura (ADRs) e wikis internas são ingeridos como fontes, e os diagramas e descrições técnicas são gerados de forma assistida, com rastreabilidade às fontes.

1.2 Escopo

Estão no escopo desta POC:

- Definição da topologia de infraestrutura (CDN/WAF, balanceadores, web servers, servidores de aplicação, mensageria, cache e camada de dados) para suportar alta volumetria transacional;
- Modelagem UML da aplicação: diagramas de casos de uso, componentes, sequência, classes e implantação;
- Documentação da arquitetura atual (AS-IS) das plataformas de IA da empresa: LLMs, Agentes de IA, RAG, n8n e MCP;
- Passo a passo do uso do NotebookLM para gerar diagramas e documentação de arquitetura;
- Faseamento da POC, critérios de sucesso, riscos e referências bibliográficas.

Estão fora de escopo: implementação de código de produção, contratação de infraestrutura e homologação regulatória formal junto ao Banco Central (esta POC produz os artefatos que subsidiarão essas etapas futuras).

1.3 Premissas e restrições

- Premissa de volume: pico estimado de 10.000 transações por segundo (TPS) em eventos de mercado, com crescimento de 40% ao ano;
- Premissa regulatória: conformidade com LGPD, resoluções do BACEN e requisitos de trilha de auditoria para operações financeiras;
- Restrição tecnológica: os microsserviços atuais do banco rodam em Kubernetes; a solução deve ser compatível com esse padrão;
- Restrição de custo: a POC deve utilizar, sempre que possível, as plataformas já contratadas (LLMs, n8n, stack de observabilidade existente);
- Restrição de dados sensíveis: nenhum dado real de cliente será ingerido no NotebookLM; apenas artefatos técnicos anonimizados.

2. Requisitos da aplicação de investimentos

2.1 Requisitos funcionais

A aplicação de investimentos do Bank OnlineX deve permitir ao cliente: consultar catálogo de produtos (CDBs, fundos, ações, tesouro direto e cripto); simular rentabilidade; emitir ordens de compra e venda; acompanhar carteira, posições e proventos em tempo quase real; receber recomendações personalizadas geradas por IA; e movimentar recursos entre conta corrente e conta de investimentos. Para a área de negócio, deve oferecer dashboards de liquidez, limites e exposição de risco.

2.2 Requisitos não-funcionais (alta transação de dados)

Os requisitos não-funcionais abaixo direcionam todas as decisões de topologia descritas na Seção 3.

ID	Requisito não-funcional	Meta (SLO)	Estratégia arquitetural
RNF-01	Disponibilidade	99,95% (≈ 4h22min de indisponibilidade/ano)	Multi-AZ, clusters ativo-ativo, health checks e failover automático
RNF-02	Throughput	10.000 TPS sustentados; 25.000 TPS de pico	Auto scaling horizontal, processamento assíncrono via Kafka
RNF-03	Latência	P95 < 300 ms em consultas; P99 < 800 ms em ordens	Cache Redis, réplicas de leitura, particionamento de dados
RNF-04	Escalabilidade	Crescer 40% ao ano sem re-arquitetura	Microsserviços stateless, event-driven, Kubernetes HPA
RNF-05	Segurança	Criptografia em trânsito (TLS 1.3) e em repouso (AES-256); zero trust	WAF, mTLS interno, HSM para chaves, Vault para segredos
RNF-06	Auditabilidade	100% das ordens com trilha imutável	Event sourcing no Kafka + trilha WORM para regulatório
RNF-07	Observabilidade	MTTD < 5 min; MTTR < 30 min	OpenTelemetry, Prometheus/Grafana, ELK e alertas SLO-based
RNF-08	Recuperação (DR)	RPO ≤ 5 min; RTO ≤ 60 min	Réplicas cross-region, backups contínuos, runbooks de DR

3. Arquitetura proposta (TO-BE) — topologia de servidores

3.1 Visão geral da topologia

A arquitetura proposta adota o padrão de microsserviços orientados a eventos (event-driven architecture) implantados em Kubernetes, distribuídos em no mínimo duas zonas de disponibilidade. O fluxo de uma requisição típica atravessa: borda (CDN/WAF), balanceador de carga L7, API Gateway (autenticação, autorização e rate limiting), camada de web servers (conteúdo estático e BFF), camada de servidores de aplicação (regras de negócio) e, por fim, mensageria e dados. A Figura 1 apresenta o diagrama de implantação.

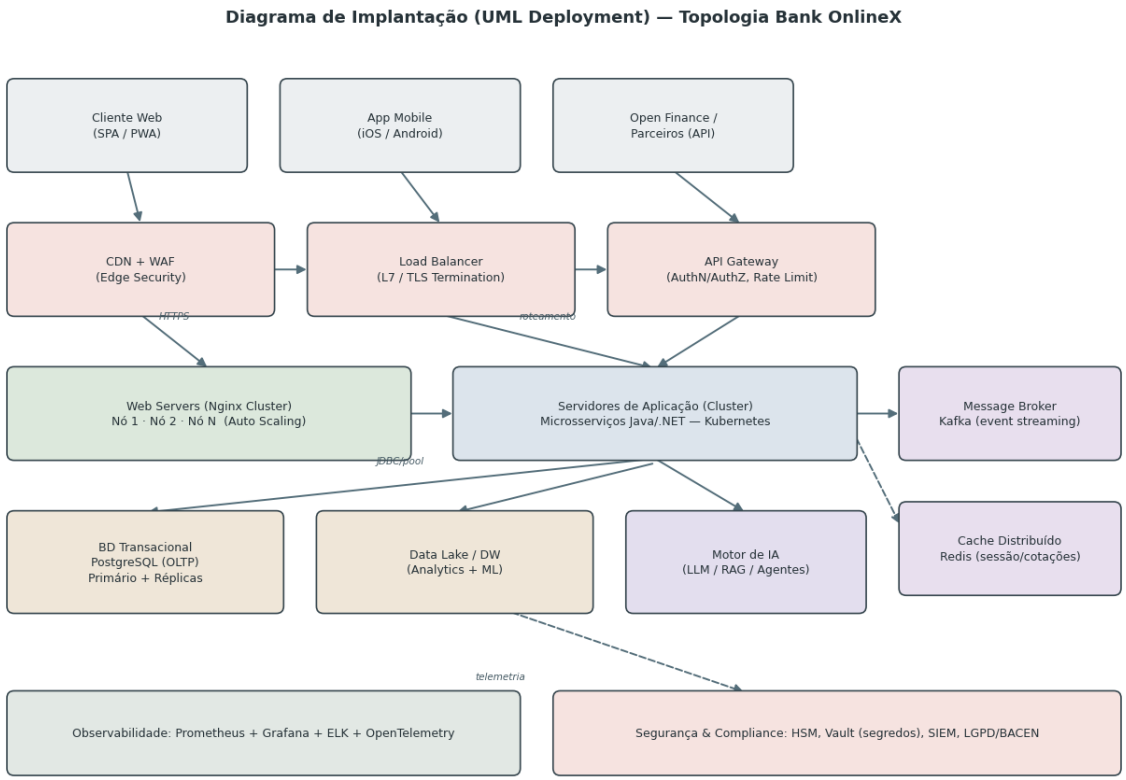


Figura 1 — Diagrama de implantação UML da topologia de destino do Bank OnlineX.

3.2 Componentes da topologia

Camada	Componente	Função na arquitetura	Observações de alta transação
Borda / Segurança	CDN + WAF	Entrega de conteúdo estático na borda; mitigação de DDoS, OWASP Top 10 e bots	Absorve picos de leitura sem atingir a origem
Borda / Tráfego	Load Balancer L7	Terminação TLS, roteamento por host/path, health checks	Escalável e redundante em múltiplas AZs
Acesso a APIs	API Gateway	AuthN/AuthZ (OIDC/JWT), rate limiting, versionamento de APIs	Protege os serviços de rajadas; quota por cliente
Apresentação	Web Servers (Nginx)	Serve o SPA/PWA e atua como reverse proxy do BFF	Stateless; auto scaling por CPU/latência

Camada	Componente	Função na arquitetura	Observações de alta transação
Aplicação	Servidores de Aplicação (Kubernetes)	Microsserviços de negócio: ordens, carteira, cotações, conta, risco	Pods stateless, HPA por TPS e latência, rolling updates
Mensageria	Apache Kafka	Backbone de eventos (event sourcing), desacoplamento e absorção de picos	Partições por chave de cliente; retenção para replay
Cache	Redis Cluster	Sessões, cotações quentes, idempotência de ordens	TTL curto em market data; pub/sub para websockets
Dados OLTP	PostgreSQL	Dados transacionais com particionamento e réplicas de leitura	Pool de conexões (PgBouncer); leituras em réplicas
Dados Analíticos	Data Lake / DW	Analytics, treinamento de modelos e relatórios regulatórios	Ingestão via CDC (Debezium) a partir do Kafka
Inteligência	Motor de IA (LLM/RAG/Agentes)	Recomendações, assistente do cliente e automações	Detalhado na Seção 4; acesso via MCP
Transversal	Observabilidade	Métricas, logs e traces correlacionados (OpenTelemetry)	Dashboards de SLO e alertas de saturação
Transversal	Segurança e Compliance	HSM, Vault, SIEM, trilhas de auditoria (LGPD/BACEN)	Zero trust com mTLS entre serviços

3.3 Decisões de arquitetura (ADRs)

- ADR-001 — Event-driven com Kafka: ordens e posições trafegam como eventos imutáveis, garantindo absorção de picos, replay e trilha de auditoria nativa (atende RNF-02 e RNF-06).
- ADR-002 — Microsserviços stateless em Kubernetes: o estado transacional fica na camada de dados e no Kafka, permitindo escalar pods horizontalmente sem afinidade de sessão (RNF-04).
- ADR-003 — API Gateway como ponto único de política: autenticação, limitação de taxa e roteamento centralizados, simplificando os serviços internos.
- ADR-004 — CQRS leve na consulta de carteira: escritas no OLTP, leituras otimizadas em réplicas e cache, reduzindo latência de consulta (RNF-03).
- ADR-005 — IA desacoplada via MCP: o motor de IA não acessa bancos diretamente; todo acesso a ferramentas e dados ocorre por servidores MCP governados (ver Seção 4.5).

4. Arquitetura atual das plataformas de IA (AS-IS)

O Bank OnlineX já opera um ecossistema de inteligência artificial composto por cinco plataformas interligadas. A Figura 2 sintetiza essa arquitetura atual, e as subseções descrevem cada componente, seu papel e como se comunica com os demais.

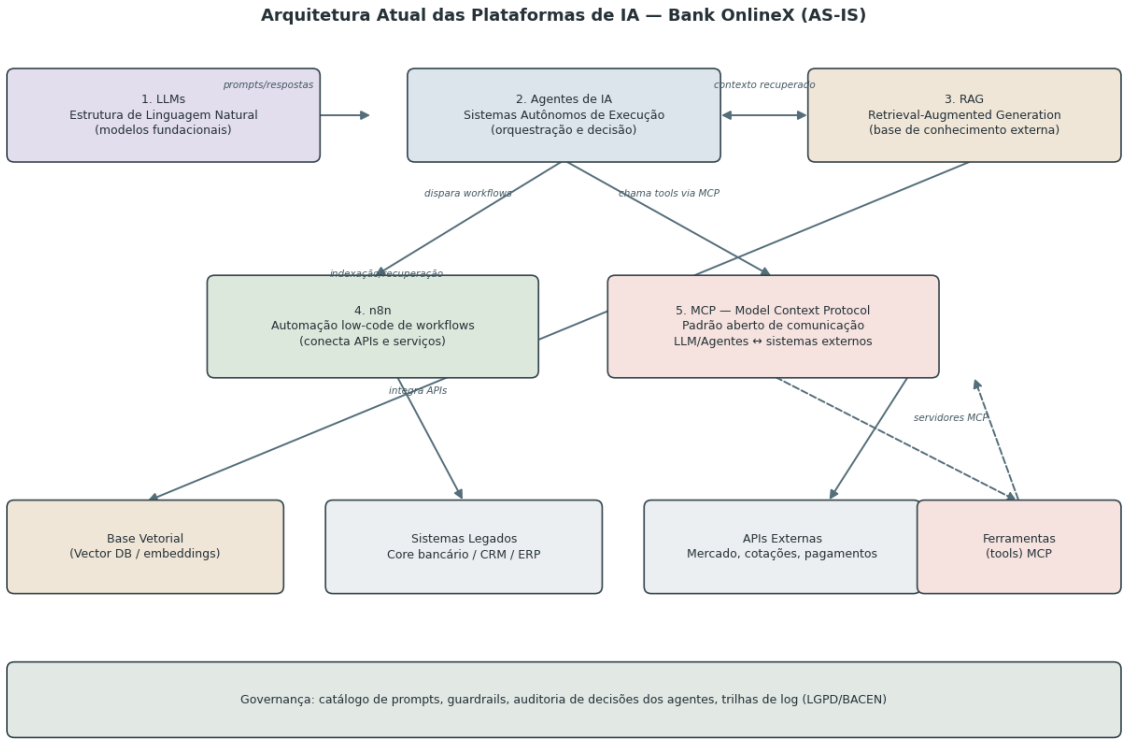


Figura 2 — Arquitetura atual das plataformas de IA do Bank OnlineX.

4.1 LLMs — Estrutura de Linguagem Natural

Os Large Language Models (LLMs) são a camada de compreensão e geração de linguagem natural da empresa. Hoje são utilizados para atendimento ao cliente (assistente virtual), sumarização de relatórios de mercado e apoio interno às equipes de produto e compliance. Os modelos são consumidos como serviço (API gerenciada), com camada de guardrails para filtragem de conteúdo sensível e políticas de não retenção de dados pessoais nos provedores.

- Entradas: prompts estruturados, instruções de sistema e contexto recuperado (RAG);
- Saídas: respostas em linguagem natural, JSON estruturado e chamadas de ferramenta (function calling);
- Governança: catálogo de prompts versionado, avaliação contínua de qualidade e registro de decisões.

4.2 Agentes de IA — Sistemas Autônomos de Execução

Os Agentes de IA são sistemas autônomos de execução que combinam um LLM como núcleo de raciocínio com um loop de planejamento-ação-observação. Na arquitetura atual, agentes executam tarefas como: monitorar carteiras e disparar alertas de rebalanceamento, compor relatórios diários de

mercado e executar rotinas operacionais (conciliações e verificações) com supervisão humana em pontos de decisão críticos (human-in-the-loop).

- Núcleo: LLM + políticas de planejamento (decomposição de tarefas em etapas);
- Memória: curto prazo (contexto da sessão) e longo prazo (base vetorial e registros de execução);
- Controle: ações irreversíveis exigem aprovação humana; toda ação gera evento de auditoria.

4.3 RAG — Retrieval-Augmented Generation

RAG (Retrieval-Augmented Generation) é a técnica que aprimora as respostas dos LLMs fornecendo informações relevantes e atualizadas de uma base de conhecimento externa. No Bank OnlineX, documentos regulatórios, lâminas de produtos, wikis internas e este próprio material de arquitetura são fragmentados (chunking), convertidos em embeddings e indexados em um banco vetorial. A cada pergunta, os trechos mais relevantes são recuperados e injetados no prompt, reduzindo alucinações e permitindo citar a fonte de cada afirmação.

- Pipeline de ingestão: extração → limpeza → chunking → embeddings → índice vetorial;
- Pipeline de consulta: pergunta → embedding → busca por similaridade → re-ranking → prompt enriquecido → resposta com citações;
- Atualização: reindexação incremental sempre que uma fonte é alterada (garante informação atualizada).

4.4 n8n — Automação low-code de workflows

O n8n é a ferramenta de automação de fluxo de trabalho, de código aberto e low-code, adotada para conectar APIs e serviços, permitindo criar automações complexas sem codificação extensiva. Hoje o n8n orquestra: disparos de notificações multicanal (e-mail, push, SMS), sincronização entre CRM e mesa de operações, enriquecimento de dados de onboarding e gatilhos que invocam agentes de IA quando eventos de negócio ocorrem (por exemplo, variação brusca de um ativo na carteira do cliente).

- Padrão de uso: workflows versionados em Git, credenciais em cofre (Vault) e ambientes dev/hom/prod;
- Integração com IA: nós de LLM e webhooks que chamam agentes, com retry e dead-letter em falhas;
- Observabilidade: logs de execução por workflow e métricas exportadas para o Prometheus.

4.5 MCP — Model Context Protocol

MCP (Model Context Protocol) é um padrão aberto projetado para padronizar a comunicação entre LLMs/Agentes de IA e os sistemas externos (ferramentas e dados) que eles precisam acessar. Em vez de integrações pontuais e heterogêneas, cada sistema expõe um servidor MCP com ferramentas (tools), recursos (resources) e prompts descritos de forma padronizada; os agentes atuam como clientes MCP e descobrem dinamicamente o que cada servidor oferece.

- Padronização: um único protocolo para expor core bancário, CRM, market data e bases vetoriais aos agentes;
- Segurança: escopos de permissão por ferramenta, autenticação e registro de cada chamada;
- Papel na POC: o NotebookLM concentra o conhecimento documental; em produção, servidores MCP expõem esse conhecimento e os sistemas transacionais aos agentes, com governança.

4.6 Integração entre as plataformas

O fluxo integrado típico é: um evento de negócio (ex.: ordem executada ou variação de mercado) dispara um workflow no n8n; o workflow invoca um Agente de IA; o agente consulta a base de conhecimento via RAG, decide a ação e, quando necessário, acessa sistemas externos através de servidores MCP; o LLM compõe a resposta final ou a recomendação; e o resultado retorna ao cliente ou ao operador humano para aprovação. Toda a cadeia emite eventos de auditoria, atendendo ao RNF-06.

5. Documentação UML

Esta seção apresenta a modelagem UML da aplicação, gerada de forma assistida pelo NotebookLM (Seção 6) e revisada pelo time de arquitetura. Foram adotados cinco diagramas: casos de uso, componentes, sequência, classes e implantação.

5.1 Diagrama de Casos de Uso

Os atores identificados são: Cliente (investidor pessoa física), Assessor Humano, Agente de IA (ator secundário que executa automações), Core Bancário (sistema externo de liquidação) e Provedor de Market Data. Os principais casos de uso por ator:

Ator	Casos de uso	Observações
Cliente	Consultar catálogo de produtos · Simular investimento · Emitir ordem de compra/venda · Acompanhar carteira · Resgatar aplicação · Conversar com assistente de IA	Casos de leitura devem atender RNF-03 (P95 < 300 ms)
Assessor Humano	Aprovar recomendação do agente · Ajustar perfil de risco do cliente · Revisar alertas de compliance	Ponto de human-in-the-loop do fluxo de IA
Agente de IA	Gerar recomendação personalizada · Monitorar carteira e rebalancear · Notificar eventos de mercado	Toda ação irreversível exige aprovação (4.2)
Core Bancário	Reservar saldo · Liquidar operação · Estornar lançamento	Integração síncrona com timeout e compensação
Market Data	Fornecer cotações em tempo real	Consumo via streaming; cache Redis como buffer

5.2 Diagrama de Componentes

A Figura 3 apresenta o diagrama de componentes, organizado em três contêineres: apresentação, serviços e dados/integração. As interfaces expostas seguem o estilo REST/HTTPS para comunicação síncrona e tópicos Kafka para comunicação assíncrona; o acesso da camada de IA a dados e ferramentas ocorre exclusivamente via MCP.

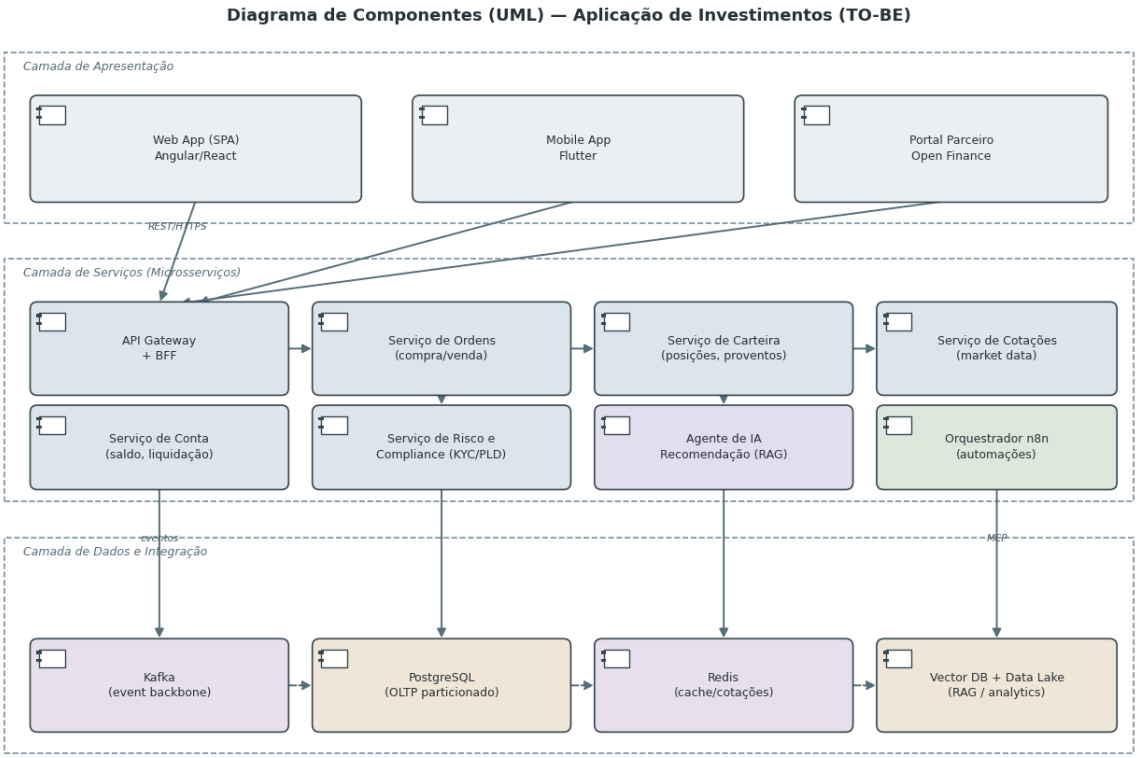


Figura 3 — Diagrama de componentes UML da aplicação de investimentos.

Descrições técnicas dos componentes principais:

- API Gateway + BFF: expõe APIs versionadas, aplica políticas de segurança e agrega chamadas para o frontend (Backend-for-Frontend), reduzindo chatter na rede móvel;
- Serviço de Ordens: recebe, valida e publica ordens como eventos; garante idempotência via chave de idempotência armazenada no Redis;
- Serviço de Carteira: consome eventos de execução e materializa posições consolidadas para leitura rápida (padrão CQRS);
- Serviço de Cotações: ingere market data por streaming, normaliza e publica em cache Redis e no Kafka para consumo dos demais serviços;
- Serviço de Risco e Compliance: valida limites, suitability (perfil do investidor) e regras de PLD/FT antes de aprovar a ordem;
- Agente de Recomendação (RAG): gera recomendações contextualizadas citando lâminas e relatórios da base de conhecimento;
- Orquestrador n8n: automações de notificação e rotinas operacionais disparadas por eventos do Kafka.

5.3 Diagrama de Sequência — execução de ordem

A Figura 4 detalha a simulação do fluxo mais crítico da aplicação: a emissão e execução de uma ordem de investimento, do clique do cliente até a confirmação e atualização da carteira, incluindo o fluxo alternativo de rejeição por saldo insuficiente.

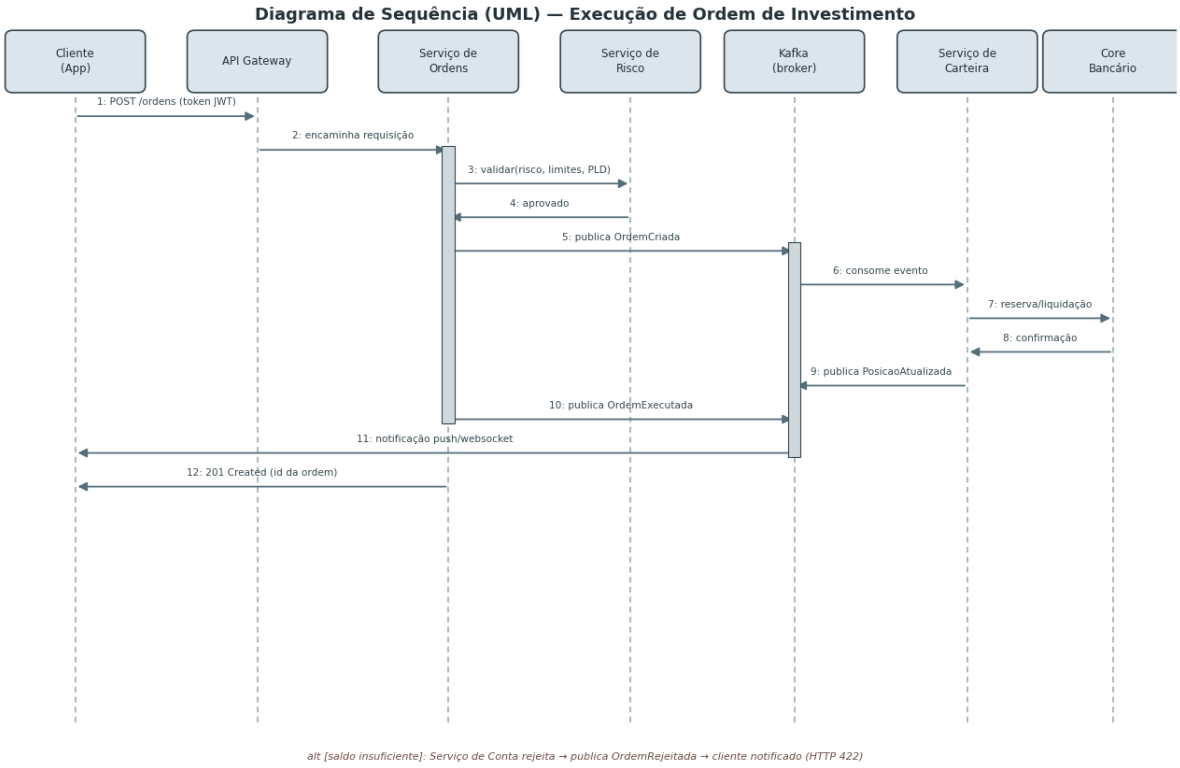


Figura 4 — Diagrama de sequência UML da execução de ordem de investimento.

Passo a passo do fluxo (conforme numeração da Figura 4):

- 1-2 — O cliente envia a ordem autenticada (JWT); o API Gateway valida o token, aplica rate limiting e encaminha ao Serviço de Ordens;
- 3-4 — O Serviço de Ordens solicita validação síncrona ao Serviço de Risco (limites, suitability, PLD); em caso de aprovação, prossegue;
- 5 — A ordem é publicada no tópico Kafka ordens.criadas, tornando-se um evento imutável e auditável;
- 6-8 — O Serviço de Carteira consome o evento, solicita reserva/liquidação ao Core Bancário e recebe a confirmação;
- 9-10 — São publicados os eventos de posição atualizada e de ordem executada, que alimentam leituras, analytics e notificações;
- 11-12 — O cliente é notificado por push/websocket e recebe a resposta síncrona HTTP 201 com o identificador da ordem;
- Fluxo alternativo (alt): se o saldo for insuficiente, o Core Bancário rejeita, publica-se o evento OrdemRejeitada e o cliente recebe HTTP 422 com o motivo — sem estado parcial, pois a compensação é orientada a eventos.

5.4 Diagrama de Classes (modelo de domínio)

O modelo de domínio central é descrito abaixo; as classes seguem os agregados do Domain-Driven Design adotado pelos microsserviços.

Classe (agregado)	Atributos principais	Relacionamentos	Serviço dono
Cliente	id, nome, cpf (anon.), perfilRisco, statusKYC	1—N ContaInvestimento; 1—N Ordem	Serviço de Conta
ContaInvestimento	id, saldoDisponivel, saldoReservado	1—N Posicao; pertence a Cliente	Serviço de Conta
Ordem	id, tipo (compra/venda), ativo, quantidade, precoLimite, status, idempotencyKey	referencia Cliente e Ativo; gera EventoOrdem	Serviço de Ordens
Ativo	ticker, classe (ação, CDB, fundo…), risco	N—N Produto; cotado por Cotacao	Serviço de Cotações
Posicao	ativo, quantidadeMedia, precoMedio, proventos	pertence a ContaInvestimento	Serviço de Carteira
Recomendacao	id, tese (via RAG), score, aprovacaoHumana	associada a Cliente e Ativo	Agente de Recomendação

5.5 Diagrama de Implantação

O diagrama de implantação é a própria Figura 1 (Seção 3.1), que mapeia os artefatos de software (microsserviços, bancos, broker) nos nós de infraestrutura (clusters Kubernetes, servidores de banco gerenciados, zonas de disponibilidade). A correspondência nó-artefato garante que nenhum componente stateful fique em nó efêmero e que todos os nós tenham redundância mínima de dois por zona.

6. Metodologia: uso do NotebookLM na construção da arquitetura

O NotebookLM é uma plataforma de pesquisa e escrita assistida por IA do Google, cujo diferencial é o grounding: as respostas são geradas exclusivamente a partir das fontes carregadas no notebook, com citações apontando para o trecho exato de cada documento. Nesta POC, ele atua como uma camada de RAG corporativa sobre os artefatos de arquitetura do Bank OnlineX, permitindo gerar desenhos e documentação UML com rastreabilidade.

6.1 Passo a passo da simulação no NotebookLM

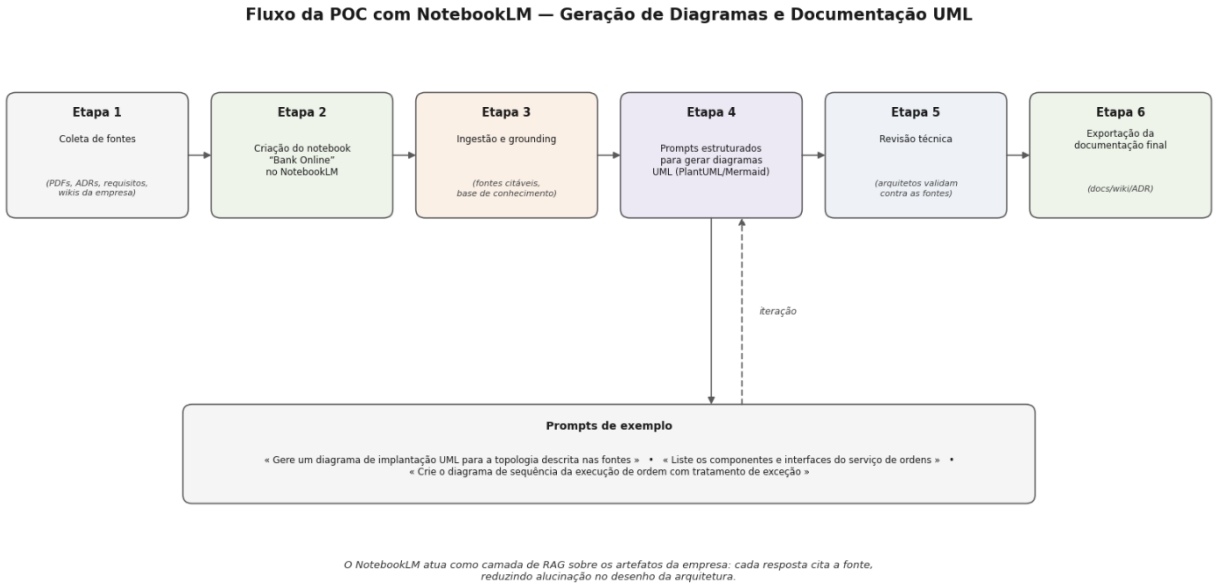


Figura 5 — Fluxo da POC com NotebookLM, da coleta de fontes à exportação da documentação.

Etapa 1 — Coleta de fontes. Reunir os artefatos de entrada: documento de requisitos de negócio, ADRs vigentes, inventário de integrações, catálogo de APIs, políticas de segurança, normas do BACEN aplicáveis e wikis de engenharia. Todos anonimizados (sem dados pessoais de clientes).

Etapa 2 — Criação do notebook. Criar no NotebookLM um notebook chamado «Bank OnlineX — Arquitetura de Investimentos» e carregar as fontes (PDF, Google Docs, Markdown e páginas web). Limite prático: manter as fontes versionadas e nomeadas por domínio (ex.: RNF.md, ADR-001-kafka.md).

Etapa 3 — Ingestão e grounding. O NotebookLM indexa as fontes e passa a responder apenas com base nelas, exibindo citações. Valida-se a cobertura perguntando: «Quais são os requisitos não-funcionais de latência e throughput?» — cada resposta deve citar RNF.md.

Etapa 4 — Geração assistida dos diagramas UML. Solicitar ao NotebookLM a geração dos diagramas em notação PlantUML ou Mermaid (ex.: «Gere um diagrama de implantação UML da topologia descrita nas fontes, com web servers, servidores de aplicação, Kafka, Redis e PostgreSQL»). Os scripts gerados são renderizados nas ferramentas correspondentes e revisados.

Etapa 5 — Revisão técnica. Os arquitetos confrontam cada diagrama com as fontes citadas, corrigem divergências e registram novas decisões (que retroalimentam o notebook como novas fontes, criando um ciclo virtuoso de documentação viva).

Etapa 6 — Exportação e publicação. A documentação consolidada é exportada para o formato oficial (este documento), publicada na wiki corporativa e referenciada nos repositórios como ADR de arquitetura de destino.

6.2 Prompts recomendados para geração de UML

Objetivo	Prompt (exemplo)	Saída esperada
Casos de uso	«Com base nos requisitos das fontes, liste atores e casos de uso e gere o diagrama de casos de uso em PlantUML»	Script PlantUML + tabela ator/caso
Componentes	«Descreva os componentes, interfaces providas e requeridas do serviço de ordens e gere o diagrama de componentes»	Diagrama de componentes + descrições
Sequência	«Crie o diagrama de sequência da execução de uma ordem, incluindo o fluxo alternativo de saldo insuficiente»	Diagrama de sequência com alt/loop
Implantação	«Gere o diagrama de implantação da topologia multi-AZ com web servers, app servers, Kafka, Redis e bancos»	Diagrama de nós e artefatos
RNF e trade-offs	«Compare as opções de mensageria citadas nas fontes e justifique a escolha registrada no ADR-001»	Análise de trade-offs citando fontes

6.3 Boas práticas e limitações

- Boas práticas: manter fontes pequenas e temáticas; nomear fontes por domínio; exigir citações nas respostas; versionar os prompts junto aos ADRs; usar a revisão humana como gate obrigatório antes de publicar;
- Limitações: o NotebookLM não substitui ferramentas de modelagem (PlantUML, draw.io, Enterprise Architect) — ele gera rascunhos rastreáveis que devem ser refinados; a qualidade depende diretamente da completude das fontes;
- Segurança: não carregar dados pessoais, segredos ou código proprietário sensível; utilizar conta corporativa com as políticas de dados do Google Workspace vigentes no banco.

7. Etapas da POC (faseamento e cronograma)

7.1 Fases, atividades e entregáveis

A POC é dividida em seis etapas sequenciais, com marcos de go/no-go ao final de cada uma. Duração total estimada: 10 semanas.

Etapa	Atividades	Entregáveis	Duração	Critério de go/no-go
1. Preparação	Coleta e anonimização das fontes; criação do notebook no NotebookLM; definição de KPIs	Notebook populado; plano de POC	1 semana	Cobertura de fontes validada por grounding
2. Modelagem UML	Geração assistida dos 5 diagramas; revisão pelos arquitetos	Pacote UML v1.0 (este documento, Seção 5)	2 semanas	Diagramas aprovados em design review
3. Topologia e sizing	Dimensionamento de clusters, partições Kafka e bancos; definição multi-AZ	Diagrama de implantação + planilha de sizing	1 semana	Sizing atende RNF-01/02 com 30% de folga
4. Protótipo de carga	Implementação mínima do fluxo de ordens (gateway → ordens → Kafka → carteira) e teste de carga	Protótipo funcional + relatório de carga	3 semanas	10.000 TPS com P99 < 800 ms sustentados
5. Integração IA	Agente de recomendação com RAG sobre a base de conhecimento; acesso a dados via MCP; automação n8n de notificação	Demo ponta a ponta com IA governada	2 semanas	Recomendações com citação de fonte e aprovação humana
6. Consolidação	Lições aprendidas, custos estimados, roadmap de produção	Relatório final e decisão de investimento	1 semana	Aprovação do comitê de arquitetura

7.2 Critérios de sucesso (KPIs)

- KPI-01 — Desempenho: protótipo sustenta 10.000 TPS com latência P99 abaixo de 800 ms no fluxo de ordens;
- KPI-02 — Rastreabilidade: 100% das afirmações arquiteturais do documento final citam uma fonte do notebook;
- KPI-03 — Qualidade da modelagem: zero divergências críticas entre diagramas UML e fontes após a revisão;
- KPI-04 — IA governada: toda recomendação do agente carrega citação da base e, quando irreversível, aprovação humana registrada;
- KPI-05 — Custos: estimativa de custo de produção com variação máxima de ±20% em relação ao orçamento aprovado.

8. Riscos e mitigação

Risco	Probabilidade	Impacto	Mitigação
Pico de mercado excede o sizing do protótipo	Média	Alto	Teste de carga com 2,5× o pico nominal; auto scaling com limites de saturação alertados
Alucinação de LLM em recomendações de investimento	Média	Alto	RAG com citação obrigatória, guardrails, aprovação humana e adequação ao perfil (suitability) antes de exibir
Vazamento de dados sensíveis em plataformas de IA	Baixa	Muito alto	Anonimização de fontes, políticas de não retenção, contas corporativas e DLP
Acoplamento excessivo ao Kafka (vendor lock-in lógico)	Baixa	Médio	Contratos de eventos versionados (schema registry) e camada anticorrupção nos consumidores
Complexidade operacional da malha de microsserviços	Média	Médio	Observabilidade desde o dia 1 (OpenTelemetry), SLOs por serviço e runbooks
Qualidade das fontes compromete o grounding no NotebookLM	Média	Médio	Curadoria de fontes, versionamento e revisão humana como gate (Etapa 5)

9. Referências e bibliografia

Fontes e sites de pesquisa utilizados como base conceitual desta POC:

- Google NotebookLM — site oficial e central de ajuda: notebooklm.google.com — conceitos de notebooks, fontes e respostas com grounding e citações.
- FOWLER, Martin. Patterns of Enterprise Application Architecture. Addison-Wesley. — catálogo de padrões de camadas, CQRS e integração.
- NEWMAN, Sam. Building Microservices, 2ª ed. O'Reilly Media. — microsserviços, comunicação síncrona/assíncrona e observabilidade.
- Object Management Group. OMG Unified Modeling Language (UML) Specification, v2.5.1. omg.org/spec/UML — semântica dos diagramas de casos de uso, componentes, sequência, classes e implantação.
- LEWIS, P. et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020. arxiv.org/abs/2005.11401 — fundamento técnico do RAG.
- Anthropic. Model Context Protocol (MCP) — especificação e documentação: modelcontextprotocol.io — padrão aberto de comunicação entre LLMs/agentes e sistemas externos.
- n8n — documentação oficial: docs.n8n.io — automação low-code de workflows, webhooks e integrações com LLMs.
- Apache Kafka — documentação oficial: kafka.apache.org/documentation — tópicos, partições, consumer groups e event streaming.
- Kubernetes — documentação oficial: kubernetes.io/docs — deployments, HPA e arquitetura de clusters.
- The Open Group. TOGAF Standard, 10ª ed. — método de arquitetura corporativa e ADRs.
- RICHARDS, Mark; FORD, Neal. Fundamentals of Software Architecture. O'Reilly Media. — trade-offs arquiteturais e características (-ilities).
- Banco Central do Brasil (BACEN) — normativos de segurança cibernética e processamento de dados para instituições financeiras: bcb.gov.br.
- Lei nº 13.709/2018 (LGPD) — proteção de dados pessoais: planalto.gov.br.

Nota metodológica: os diagramas desta POC foram gerados de forma assistida a partir das fontes listadas, conforme o processo da Seção 6, e revisados pela equipe de arquitetura do Bank OnlineX.